

Algorithms

Chapter 9

Medians and Order Statistics

中位數

順序統計

Associate Professor: Ching-Chi Lin

林清池 副教授

chingchi.lin@gmail.com

Department of Computer Science and Engineering
National Taiwan Ocean University

Outline

- ▶ **Minimum and maximum**
- ▶ Selection in expected linear time
- ▶ Selection in worst-case linear time

Order statistics

- ▶ The **i th order statistic** of a set of n elements is the i th smallest element. 第 i 個順序統計值就是第 i 小元素
- ▶ The **minimum** is the first order statistic ($i = 1$).
- ▶ The **maximum** is the n th order statistic ($i = n$).
- ▶ A **median** is the “halfway point” of the set. 集合中途點 = 中位數
- ▶ When n is odd, the median is unique, at $i = (n + 1)/2$. 1 . 2 . 3 . 4 . 5
 ↑
 median
- ▶ When n is even, there are two medians: 1 . 2 . 3 . 4 . 5 . 6 lower upper
 ↑ ↑
 lower median
- ▶ The **lower median**: $i = \lfloor (n + 1)/2 \rfloor$, and
- ▶ The **upper median**: $i = \lceil (n + 1)/2 \rceil$.
- ▶ We mean lower median when we use the phrase “the median”.
沒有指定, median = lower median

The selection problem 找第 i 小

- ▶ How can we find the i th order statistic of a set and what is the running time?
- ▶ **Input:** A set A of n (distinct) number and a number i , with $1 \leq i \leq n$.
- ▶ **Output:** The element $x \in A$ that is larger than exactly $i-1$ other elements of A .
- ▶ The **selection problem** can be solved in **$O(n \lg n)$** time.
 - ▶ Sort the numbers using an $O(n \lg n)$ -time algorithm, such as heapsort or merge sort. 先排序(由小到大)
 - ▶ Then return the i th element in the sorted array. 找第 i 個
- ▶ Are there faster algorithms?
 - ▶ An $O(n)$ -time algorithm would be presented in this chapter.

Finding minimum 找最小

- ▶ We can easily obtain an upper bound of $n-1$ comparisons for finding the minimum of a set of n elements. 一個一個比，保留最小，共比 $n-1$ 次
- ▶ Examine each element in turn and keep track of the smallest one.
- ▶ The algorithm is optimal, because each element, except the minimum, must be compared to a smaller element at least once.

MINIMUM(A)

演算法最佳，每個元素至少跟最小比過一次

1. $min \leftarrow A[1]$
2. **for** $i \leftarrow 2$ **to** $length[A]$
3. **do if** $min > A[i]$
4. **then** $min \leftarrow A[i]$
5. **return** min

- ▶ The maximum can be found in exactly the same way by replacing the $>$ with $<$ in the above algorithm. 同樣方法可找最大

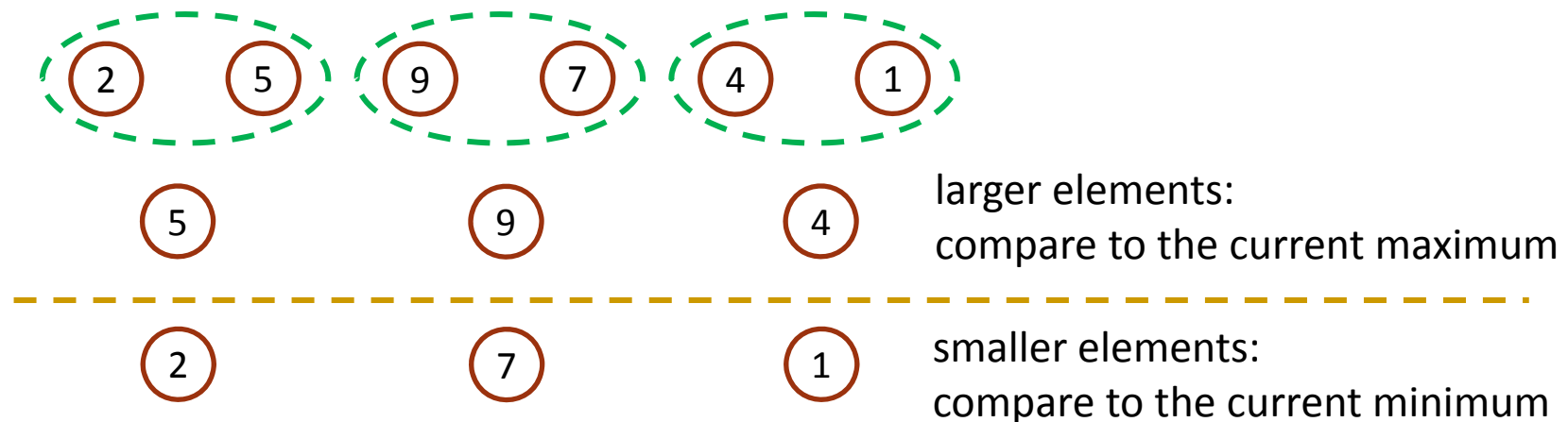
同時找最小和最大

Simultaneous minimum and maximum_{1/3}

- ▶ Some applications need both the minimum and maximum.
 - ▶ Find the minimum and maximum independently, using $n-1$ comparisons for each, for a total of $2n-2$ comparisons.
較差的方法, 分別找一次最小和最大
 - ▶ In fact, at most $3 \lfloor n/2 \rfloor$ comparisons are needed: 保留已知最大最小
 - ▶ Maintain the minimum and maximum of elements seen so far.
 - ▶ Process elements in pairs. 一次處理兩個
 - ▶ Compare the elements of a pair to each other. 兩個先互比
 - ▶ Then compare the larger element to the maximum so far, and compare the smaller element to the minimum so far.
較大的跟目前最大者比, 較小的跟目前最小者比
 - ▶ This leads to only 3 comparisons for every 2 elements.
每兩個只需比三次
- 2个元素互比, 然後: 較大者跟目前最大的比 > 維護最大-最小
較小者跟目前最小的比

Simultaneous minimum and maximum_{2/3}

- ▶ An observation 因為較大不會是最小，較小不會是最大
 - ▶ If we compare the elements of a pair to each other, the larger can't be the minimum and the smaller can't be the maximum.
 - ▶ So we just need to compare the larger to the current maximum and the smaller to the current minimum. 所以較大跟最大比，較小跟最小比
 - ▶ It costs 3 comparisons for every 2 elements. 每兩個比3次
 - ▶ The previous method costs 2 comparisons for each element.



Simultaneous minimum and maximum_{3/3}

- ▶ Setting up the initial values for the min and max depends on whether n is odd or even.
 - ▶ If n is even, compare the first two elements and assign the larger to max and the smaller to min.
 - ▶ If n is odd, set both min and max to the first element.
- ▶ If n is even, # of comparisons $= \frac{3(n-2)}{2} + 1 = \frac{3n}{2} - 2$.
- ▶ If n is odd, # of comparisons $= \frac{3(n-1)}{2} = 3\lfloor n/2 \rfloor$.
- ▶ In either case, the # of comparisons is $\leq 3\lfloor n/2 \rfloor$.

若 n 為偶數,找兩數令大者為最大,小者為最小

若 n 為奇數,找任一數同時為最大和最小

Outline

- ▶ Minimum and maximum
- ▶ **Selection in expected linear time**
- ▶ Selection in worst-case linear time

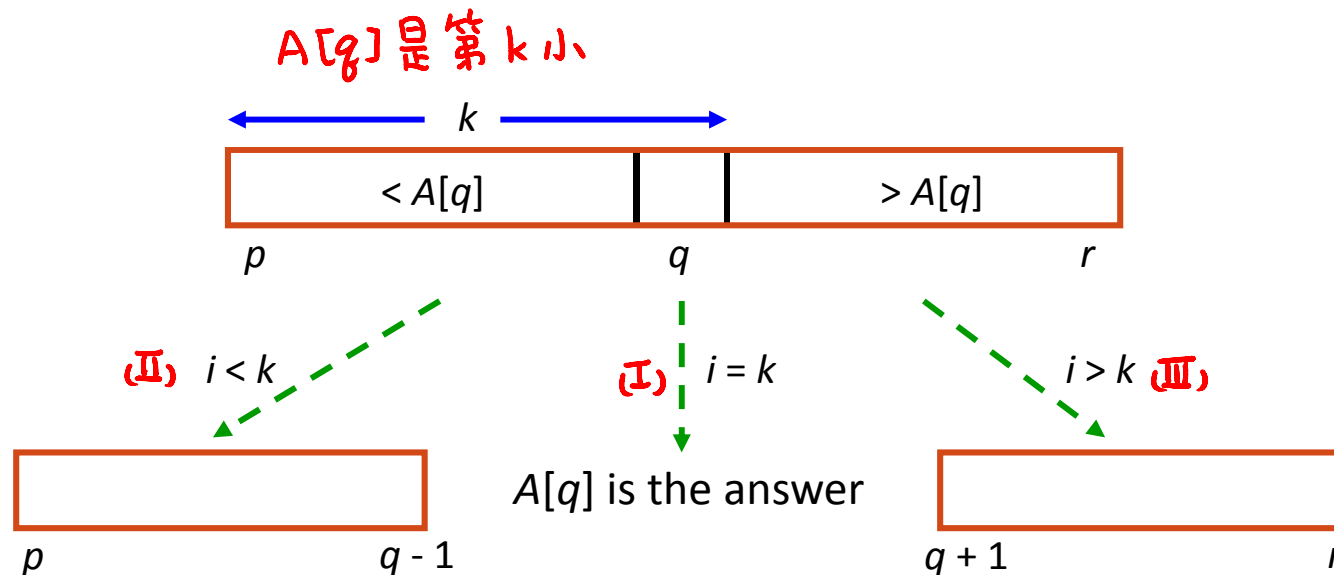
Selection in expected linear time

- ▶ In fact, selection of the i th smallest element of the array A can be done in $\Theta(n)$ time. 找第 i 小需要 $\Theta(n)$ 時間
- ▶ We first present a randomized version in this section and then present a deterministic version in the next section.
此節是隨機性版本, 下一節是確定性版本, 都是 $O(n)$ 時間
- ▶ The function RANDOMIZED-SELECT:
 - ▶ is a divide-and-conquer algorithm,
 - ▶ uses RANDOMIZED-PARTITION from the quicksort algorithm in Chapter 7, and
 - ▶ recurses on one side of the partition only.
只會在某一邊遞迴

RANDOMIZED-SELECT procedure_{1/2}

1. RANDOMIZED-SELECT(A, p, r, i)
2. **if** $p = r$
3. **then return** $A[p]$ 只有一個, 直接回傳
4. $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$
5. $k \leftarrow q - p + 1$
6. **if** $i = k$ /* the pivot value is the answer */ Ⅰ) $i=k$, 直接回傳
7. **then return** $A[q]$ 找到了
8. **elseif** $i < k$
9. **then return** RANDOMIZED-SELECT($A, p, q - 1, i$) Ⅱ) 在左邊找
10. **else return** RANDOMIZED-SELECT($A, q + 1, r, i - k$) Ⅲ) 在右邊找

RANDOMIZED-SELECT procedure_{2/2}



To find the **i th** order statistic in
 $A[p \dots q-1]$

範圍縮小, 找第 i 小

To find the **$(i-k)$ th** order statistic in
 $A[q+1 \dots r]$

範圍縮小, 找第 $i-k$ 小

Algorithm analysis_{1/4}

- ▶ The **worst case**: always recurse on a subarray that is only 1 element smaller than the previous subarray.

- ▶ $T(n) = T(n - 1) + \Theta(n)$
 $= \Theta(n^2)$

每次只去掉一個

- ▶ The **best case**: always recurse on a subarray that has half of the elements smaller than the previous subarray.

- ▶ $T(n) = T(n/2) + \Theta(n)$
 $= \Theta(n)$ (Master Theorem, case 3)

分割平均

Algorithm analysis_{2/4}

► The **average case**:

- We will show that $T(n) = \Theta(n)$.
- For $1 \leq k \leq n$, the probability that the subarray $A[p..q]$ has k elements is $1/n$. $A[q]$ 是第 k 小 ($1 \leq k \leq n$) 机率皆為 $\frac{1}{n}$

- So, we have

$$\begin{aligned} T(n) &\leq \sum_{k=1}^n \frac{1}{n} \cdot (T(\max(k-1, n-k)) + O(n)) \\ &= \sum_{k=1}^n \frac{1}{n} \cdot T(\max(k-1, n-k)) + O(n). \end{aligned}$$

遞迴時間 要找的在個數較多那邊 partition 時間

$n=100$, k 從 1~100 出現一次 $\Leftrightarrow T(k)$ 從 50~99 出現二次

$n=101$, k 從 1~101 出現一次 $\Leftrightarrow T(k)$ 從 51~100 出現二次, 50 出現一次

Algorithm analysis_{3/4}

相加取期望 = 個別取期望

再相加

$$E[T(n)] \leq E\left[\sum_{k=1}^n \frac{1}{n} \cdot T(\max(k-1, n-k)) + O(n)\right] = \sum_{k=1}^n E\left[\frac{1}{n} T(\max(k-1, n-k))\right] + O(n)$$

$$= \sum_{k=1}^n \frac{1}{n} \cdot E[T(\max(k-1, n-k))] + O(n) \leq \frac{2}{n} \sum_{k=\lceil n/2 \rceil}^{n-1} E[T(k)] + O(n).$$

乘常數取期望值 = 取期望後再乘常數

$$\because \max(k-1, n-k) = \begin{cases} k-1 & \text{if } k > \lceil n/2 \rceil \\ n-k & \text{if } k \leq \lceil n/2 \rceil \end{cases}$$

k	1	2	...	$\lceil n/2 \rceil$	$\lceil n/2 \rceil + 1$	...	$n-1$	n
$\max(k-1, n-k)$	$n-1$	$n-2$	...	$n - \lceil n/2 \rceil = \lfloor n/2 \rfloor$	$\lceil n/2 \rceil$	...	$n-2$	$n-1$

- ▶ If n is even, each term from $T(\lceil n/2 \rceil)$ to $T(n-1)$ appears exactly twice.
- ▶ If n is odd, each term from $T(\lceil n/2 \rceil)$ to $T(n-1)$ appears exactly twice and $T(\lfloor n/2 \rfloor)$ appears once.
 - ▶ Because $k = \lceil n/2 \rceil$, $k-1 = n-k = \lfloor n/2 \rfloor$.

Algorithm analysis_{4/4}

► Solve this recurrence by substitution:

- Assume $E[T(n)] \leq cn$ for sufficiently large c .
- The function described by the $O(n)$ term is bounded by an for all $n > 0$.

► Then, we have

$$\begin{aligned}
 E[T(n)] &\leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + O(n) \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + an \quad \text{目標: } \leq cn \text{ (assume } O(n)) \text{ 置換} \\
 &= \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right) + an = \frac{2c}{n} \left(\frac{(n-1)n}{2} - \frac{(\lfloor n/2 \rfloor - 1)\lfloor n/2 \rfloor}{2} \right) + an \quad \text{c 的條件} \\
 &\leq \frac{2c}{n} \left(\frac{(n-1)n}{2} - \frac{(n/2 - 2)(n/2 - 1)}{2} \right) + an \quad \text{chose } c \text{ so that } c/4 - a > 0 \\
 &= \frac{c}{n} \left(\frac{3n^2}{4} + \frac{n}{2} - 2 \right) + an = c \left(\frac{3n}{4} + \frac{1}{2} - \frac{2}{n} \right) + an \\
 &\leq \frac{3cn}{4} + \frac{c}{2} + an = \underline{cn - \left(\frac{cn}{4} - \frac{c}{2} - an \right)} \leq cn \quad \text{no 的條件}
 \end{aligned}$$

$$\begin{aligned}
 \frac{cn}{4} - \frac{c}{2} - an &\geq 0 \\
 n\left(\frac{c}{4} - a\right) &\geq \frac{c}{2} \\
 n &\geq \frac{2c}{c - 4a}
 \end{aligned}$$

► Thus, if we assume that $T(n) = O(1)$ for $n < 2c/(c - 4a)$, we have $E[T(n)] = O(n)$.

Outline

平均而言 linear time, worst-case 為 $O(n^2)$

- ▶ Minimum and maximum
- ▶ Selection in expected linear time
- ▶ **Selection in worst-case linear time**

SELECT algorithm_{1/2}

- ▶ **Idea:** Guarantee a good split when the array is partitioned.

保證有好分割, 當左右有一定比例

- ▶ The SELECT algorithm: $\rightarrow$ 可消除右或左

1. Divide n elements into groups of 5 elements. 分 5 個一組

2. Find median of each of the $\lceil n/5 \rceil$ groups. 找每組中位數

- ▶ Run insertion sort on each group.

- ▶ Then just pick the median from each group.

用遞迴找出所有中位數的中位數

3. Use SELECT recursively to find median x of the $\lceil n/5 \rceil$ medians.

4. Partition the n elements around x .

- ▶ Let x be the k th element of the array after partitioning.

- ▶ There are $k - 1$ elements on the low side of the partition and $n - k$ elements on the high side.

用找出的中位數的中位數, 將陣列分兩邊

SELECT algorithm_{2/2}

5. Now there are three possibilities:

- ▶ If $i = k$, then return x . **$i=k$, 直接回傳**
- ▶ If $i < k$, then use SELECT recursively to find i th smallest element on the low side. **在左邊找**
- ▶ If $i > k$, then use SELECT recursively to find $(i-k)$ th smallest element on the high side. **在右邊找**

雖然存在不確定的綠色區塊，但可以確定：

- 紅區必大於 x

- 灰區必小於 x

⇒ 可以刪掉一些區塊

小於 x 至少有 $\frac{3n}{10} - 6$ 個

綠色區塊不知道

每組的中位數

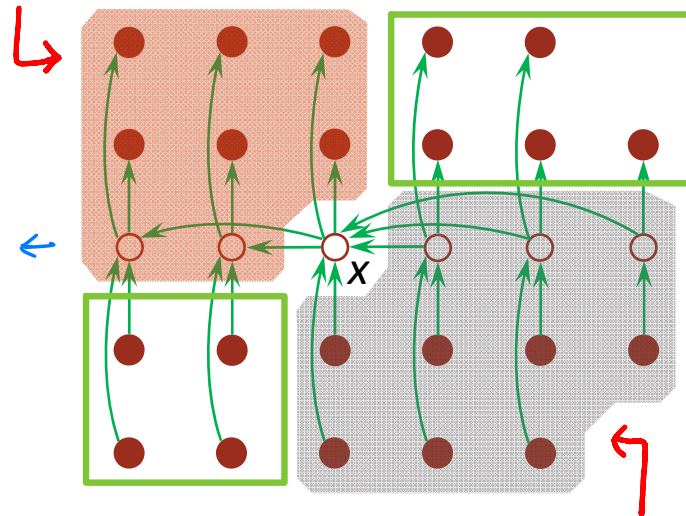
○ : The median of a group.

→ : From larger to smaller.

▨ : Elements in this region are greater than x .

▨ : Elements in this region are smaller than x .

偶數時: Lower median ←



大於 x 至少有 $\frac{3n}{10} - 6$ 個

→ 保證最壞情況還是可以踢掉30%的資料

刪完剩下需遞迴元素最多:

$\frac{7n}{10} + 6$ 個

Time complexity_{1/3}

- ▶ At least half of the medians are $\geq x$. 至少有一半的中位數大於等於 x
 - ▶ Precisely, at least $\lceil \lceil n/5 \rceil / 2 \rceil$ medians $\geq x$.
- ▶ These groups contribute 3 elements that are $> x$, except for 2 of the groups: 每組至少有 3 個元素大於 x , 除了
 - ▶ the group containing x , and
 - ▶ the group with < 5 elements.
- ▶ The number of elements greater than x is at least:
 - ▶ $3 (\lceil \lceil n/5 \rceil / 2 \rceil - 2) \geq 3n/10 - 6$.
- ▶ Similarly, at least $3n/10 - 6$ elements are less than x .
- ▶ Thus, SELECT is called recursively on $\leq 7n/10 + 6$ elements in step 5.

Time complexity_{2/3}

▶ The SELECT algorithm:

1. Divide n elements into groups of 5 elements. $O(n)$
2. Find median of each of the $\lceil n/5 \rceil$ groups. $O(n)$
 - ▶ Run insertion sort on each group.
 - ▶ Then just pick the median from each group.
3. Use SELECT recursively to find median x of the $\lceil n/5 \rceil$ medians. $T(\lceil n/5 \rceil)$
4. Partition the n elements around x . $O(n)$
 - ▶ Let x be the k th element of the array after partitioning.
 - ▶ There are $k - 1$ elements on the low side of the partition and $n - k$ elements on the high side.
5. Now there are three possibilities: $T(7n/10 + 6)$
 - ▶ If $i = k$, then return x .
 - ▶ If $i < k$, then use SELECT recursively to find i th smallest element on the low side.
 - ▶ If $i > k$, then use SELECT recursively to find $(i - k)$ th smallest element on the high side.

▶ Time complexity: $T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n)$.

Time complexity_{3/3}

- ▶ Solve this recurrence by substitution: 用置換法分析

- ▶ Assume $T(n) \leq cn$ for sufficiently large c .
- ▶ The function described by the $O(n)$ term is bounded by an for all $n > 0$.

- ▶ Then, we have

$$\begin{aligned} T(n) &\leq T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) \\ &\leq c\lceil n/5 \rceil + c(7n/10 + 6) + an \\ &\leq cn/5 + c + 7cn/10 + 6c + an \\ &= 9cn/10 + 7c + an \\ &= cn + (-cn/10 + 7c + an) \end{aligned}$$

算出的結果

- ▶ This last quantity is $\leq cn$ if we choose $c \geq 20a$.

$$-cn/10 + 7c + an \leq 0$$

$$cn/10 - 7c \geq an$$

$$c(n - 70) \geq 10an$$

$$c \geq 10a(n/(n - 70))$$

Notice: $(n/(n-70)) \leq 2$ for $n \geq 140$.

n_0